

Nama : Akhmad Satrio Cahyo Pratama
NRP : 3125522013
Studi : Teknik Informatika

1 Sprint Goal

Menghubungkan class utama proyek (**Nasabah, Sampah, Setoran, BankSampah, Rekening**) sehingga object dapat saling berinteraksi sebagai satu sistem melalui association, aggregation, dan composition.

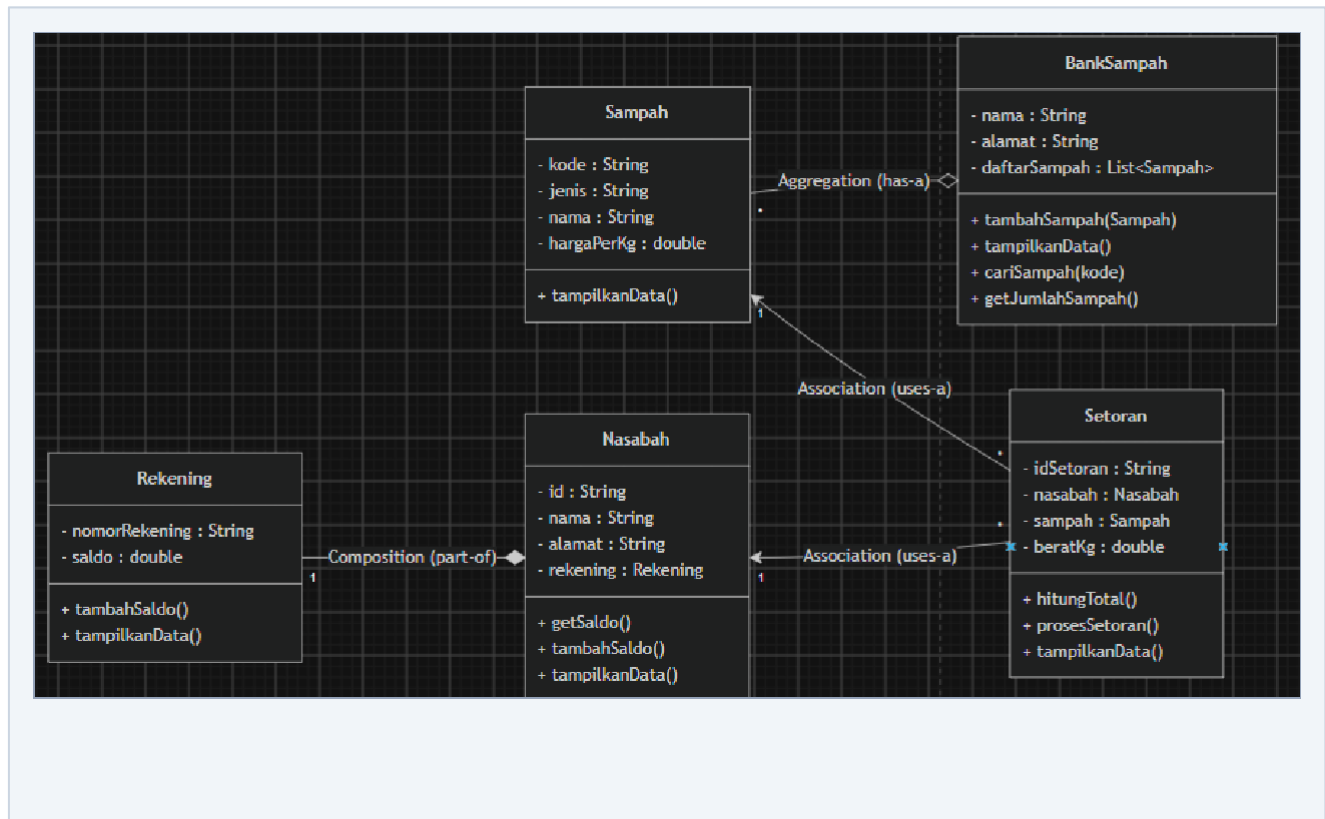
2 Sprint Backlog

ID	Sprint Backlog	Status
SB-01	Mengidentifikasi relasi antarclass	DONE
SB-02	Memperbarui class diagram	DONE
SB-03	Implementasi association (Setoran → Nasabah & Sampah)	DONE
SB-04	Implementasi aggregation (BankSampah → Sampah)	DONE
SB-05	Implementasi composition (Nasabah → Rekening)	DONE
SB-06	Membuat pengujian antarobject (Test 1, 2, 3)	DONE

3 Tabel Identifikasi Relasi

Class A	Class B	Relasi	Alasan
Setoran	Nasabah	Association	Setoran menggunakan object Nasabah untuk mencatat siapa yang menyetor (uses-a).
Setoran	Sampah	Association	Setoran menggunakan object Sampah untuk membaca jenis & harga per kg (uses-a).
BankSampah	Sampah	Aggregation	BankSampah memiliki (has-a) daftar Sampah, tetapi Sampah tetap dapat hidup mandiri.
Nasabah	Rekening	Composition	Rekening merupakan bagian (part-of) dari Nasabah; Rekening dibuat di dalam Nasabah.

4 Class Diagram Terbaru



Catatan Perbaikan Diagram:

- Mengimplementasikan arah panah (*navigability*) yang jelas untuk relasi *Association* dari **Setoran** menuju **Nasabah** dan **Sampah**.
- Posisi *multiplicity* telah dikoreksi (misal: 1 pada sisi **Nasabah** dan * pada sisi **Setoran**).
- Simbol belah ketupat untuk *Composition* dan *Aggregation* digambar rapi di luar kotak class.
- Koreksi penulisan (typo) pada method `tampilkanData()` di class **BankSampah** dan **Sampah**.

5 Penjelasan Relasi Antarobject

Association (**Setoran** → **Nasabah** dan **Setoran** → **Sampah**)

Association adalah hubungan *uses-a* yang lemah. Class **Setoran** menyimpan referensi object **Nasabah** dan **Sampah** sebagai attribute (bukan lagi string ID seperti P3). **Setoran** memanggil method object lain, misalnya `nasabah.getNama()` dan `sampah.getHargaPerKg()`. Semua object hidup mandiri; jika **Setoran** dihapus, **Nasabah** dan **Sampah** tidak terpengaruh.

Aggregation (**BankSampah** ♦-- **Sampah**)

Aggregation adalah hubungan *has-a* yang sedang. **BankSampah** memiliki daftar **Sampah** yang ditampung dalam bentuk `List<Sampah>`. Object **Sampah** dibuat di luar kemudian dimasukkan melalui method `tambahSampah(Sampah)`, sehingga **Sampah** tetap dapat berdiri sendiri (independent) meskipun object

Composition (Nasabah ↔ Rekening)

Composition adalah hubungan *part-of* yang kuat. Object **Rekening** dibuat langsung di dalam constructor **Nasabah** (contoh: `new Rekening(" R-" +id, saldoAwal)`). **Rekening** tidak dapat berdiri sendiri tanpa **Nasabah**; jika object **Nasabah** dihapus, maka **Rekening** ikut terhapus. Saldo nasabah didelegasikan sepenuhnya ke **Rekening** melalui method `tambahSaldo()` dan `getSaldo()`.

6 Skenario & Hasil Pengujian

Seluruh skenario pengujian telah berhasil dijalankan (Kompilasi sukses, output sesuai dengan skenario).

Kode	Skenario	Hasil
Test 1	Object berhasil dibuat (Sampah, Nasabah, BankSampah, Rekening, Setoran).	Berhasil - semua object tercetak lengkap.
Test 2	Dua/lebih object berinteraksi: tambah Sampah (agg), prosesSetoran mengubah saldo Nasabah (comp).	Berhasil - saldo bertambah setelah proses setoran.
Test 3	Data object lain digunakan melalui method: <code>nasabah.getNama()</code> , <code>sampah.getHarga()</code> .	Berhasil - data object lain terbaca via getter.
Validasi	Data tidak valid ditolak ($\text{berat} \leq 0$, nama kosong, nominal negatif).	Berhasil - nilai tetap, pesan error tampil.

7 Sprint Review & Retrospective

Sprint Review

- **Class diagram diperbarui:** Ya, telah menampilkan 5 class beserta multiplicity dan spesifikasi association/aggregation/composition.
- **Relasi berhasil diimplementasikan:** Association (Setoran ↔ Nasabah/Sampah), Aggregation (BankSampah has-a Sampah), dan Composition (Nasabah part-of Rekening).
- **Kendala:** Menentukan relasi aggregation vs composition. Hal ini berhasil diselesaikan dengan pertanyaan validasi: *'apakah bagian bisa hidup tanpa pemiliknya?'*

Sprint Retrospective

- **What Went Well:** Association dan aggregation berhasil diimplementasikan dengan baik karena pola pembuatan object (di dalam vs di luar class) memudahkan dalam menentukan jenis relasi.
- **What Went Wrong:** Awalnya sempat mengalami kesulitan dalam membedakan secara teknis antara aggregation dan composition; sehingga memerlukan pendalaman konsep *has-a* vs *part-of*.
- **Improvement:** Desain class ke depannya dapat ditingkatkan dengan memakai collection pada Nasabah (untuk menyimpan daftar Rekening) serta menambah relasi antarobject lain seperti penggunaan Enum untuk jenis sampah.